A FRIENDLY REVISION NOTEBOOK

DSA with Python

Handwritten-style study notes • Fundamentals to interview concepts

Definitions | Python examples | Time + space complexity | Exam patterns

How to study:
Understand the idea → trace the example → explain why it works → calculate time and space → test edge cases.

A complete revision guide covering every topic from the accompanying DSA explanation.

Notebook map

Use this table of contents to revise one topic at a time. This notebook keeps the definitions, examples, and complexity notes together so you do not have to learn everything in one sitting.

1. **Part 1:** DSA and Python foundations (Sections 1-7)
2. **Part 2:** Algorithm analysis and complexity (Sections 8-14)
3. **Part 3:** Core data structures (Sections 15-22)
4. **Part 4:** Essential algorithms (Sections 23-33)
5. **Part 5:** DSA exam problem patterns (Sections 34-37)
6. **Part 6:** Interview and correctness concepts (Sections 38-42)
7. **Final page:** Revision checklist and study order

> **Notation reminder:** `n` usually means input size; `V` means number of graph vertices; `E` means number of graph edges.

Learning loop

For each problem, answer four questions: What is the input? What is the output? What is the simplest correct logic? Can the time or extra space be improved?

PART 1

DSA and Python foundations

1. What is DSA?

DSA = Data Structures and Algorithms. A data structure is how data is stored and organized. An algorithm is a step-by-step procedure to solve a problem.

- **Data structure:** a list can store `[10, 20, 30, 40, 50]`.
- **Algorithm:** a search method checks where the value `30` occurs.

```
numbers = [10, 20, 30, 40, 50]

for num in numbers:
    if num == 30:
        print("Found")
        break
```

DSA helps us write solutions that are correct, efficient and scalable.

2. Variables and data types

A **variable** is a name that refers to a value.

```
n = 10
name = "Python"
price = 10.5
is_valid = True
```

Type	Meaning	Example
`int`	Integer	`10`
`float`	Decimal number	`10.5`
`str`	Text	`"hello"`
`bool`	True or False	`True`
`list`	Ordered, mutable collection	`[1, 2, 3]`
`tuple`	Ordered, immutable collection	`(1, 2, 3)`
`set`	Unique values	`{1, 2, 3}`
`dict`	Key-value pairs	`{"a": 10}`

Data types matter because each supports different operations and performance characteristics.

3. Operators

Category	Operators	Purpose
Arithmetic	`+ - * /`	Calculations
Floor division	`//`	Division rounded down
Modulus	`%`	Remainder
Exponentiation	`**`	Power
Comparison	`== != > <`	Compare values
Logical	`and or not`	Combine conditions
Assignment	`= += -=`	Assign/update values
Membership	`in, not in`	Check membership

```
n = 123
last_digit = n % 10
remaining = n // 10
print(last_digit)  # 3
print(remaining)   # 12
```

Remember for digit problems: for a positive integer, `n % 10` extracts the last digit and `n // 10` removes the last digit. Common uses: reverse a number, count digits, palindrome and Armstrong problems.

4. Conditions

Conditions let a program make decisions. `if` checks a condition, `elif` checks another if earlier tests were false, `else` handles the remaining case, nested conditions place a condition inside another condition, and compound conditions combine tests with `and` / `or`.

```
def even_odd(n):
    if n % 2 == 0:
        return "Even"
    else:
        return "Odd"
```

```
def eligible(age):
    if age >= 18:
        return "Eligible"
    return "Not eligible"
```

The voting example assumes that the input age is valid and nonnegative. A condition chooses the returned answer; the final `return` ends the function call.

5. Loops

Loops repeat a block of code.

`for` loop

Use it when iterating through a sequence or range.

```
for i in range(1, 6):
    print(i)
# Output: 1 2 3 4 5
```

`while` loop

Use it while a condition remains true. Example: process digits until the number becomes zero.

```
n = 123
while n > 0:
    print(n % 10)
    n //= 10
# Output: 3, then 2, then 1 (on separate lines)
```

Statement	Meaning
`break`	Exit the loop
`continue`	Skip the current iteration
`pass`	Do nothing; placeholder statement

Choose the clearest loop: `for` is convenient for a list, string, dictionary or known range. `while` is convenient when repeating until a condition changes. Many tasks can be solved either way.

6. Functions

A function is a reusable block of code that performs a task.

```
def square(a):
    return a * a

result = square(5)
print(result)  # 25
```

Term	Meaning
Function definition	`def square(a):`
Parameter	`a`, the name in the definition
Argument	`5` in `square(5)`
Function call	`square(5)`
`return`	Sends a value to the caller
`print()`	Displays output
Local variable	Defined inside a function
Global variable	Defined outside a function

```
def add(a, b):
    return a + b

result = add(10, 20) + 5
print(result)  # 35
```

`add(10, 20)` evaluates to `30`; then `30 + 5` becomes `35`. A function can return without printing. If a function reaches the end without an explicit return value, Python returns `None`.

Other function concepts: default arguments have preset values; keyword arguments are passed by parameter name; multiple return values are commonly returned as a tuple; scope determines where a variable is accessible; recursion means a function calls itself.

7. Python collections used in DSA

A. List

An ordered collection that can be modified. Indexing starts at zero; a list of length `n` has last index `n - 1`.

```
numbers = [10, 20, 30]
numbers.append(40)
numbers[0] = 5
print(numbers)  # [5, 20, 30, 40]
```

Useful methods: `append(x)` adds at the end; `insert(i, x)` inserts at an index; `pop()` removes and returns an element; `remove(x)` removes the first matching value; `len(numbers)` returns count; `numbers[i]` accesses an index; `numbers.sort()` sorts in place.

B. String

An ordered sequence of characters; strings are immutable.

```
text = "python"
print(text[0])    # p
print(text[::-1]) # nohtyp
```

Common operations include slicing, searching, counting characters and concatenation. Because strings are immutable, an individual character cannot be reassigned directly.

C. Tuple

An ordered, immutable collection. Example: `point = (10, 20)`. Useful for fixed groups of values such as coordinates or for returning multiple results.

D. Set

A collection of unique elements.

```
numbers = {1, 2, 2, 3}
print(numbers)  # {1, 2, 3}
```

Sets help remove duplicates and check membership. Average membership lookup is `O(1)`, although worst-case performance can be slower.

E. Dictionary

A dictionary stores key-value pairs and is especially useful for frequency counting, duplicate detection, grouping and lookup.

```
count = {}
numbers = [2, 2, 3]

for num in numbers:
    count[num] = count.get(num, 0) + 1

print(count)  # {2: 2, 3: 1}
```

- `num` is the key.
- `count[num]` accesses or updates the value associated with that key.
- `.get(num, 0)` returns the existing value or `0` if the key is absent.
- `+ 1` increments the frequency.

Square brackets are used to access or update a dictionary value by key. A dictionary key must be hashable; common keys include strings, integers and tuples of hashable values.

PART 2

Algorithm analysis

8. What is time complexity?

Time complexity describes how the amount of work performed by an algorithm grows as input size grows. We usually call input size `n`. A list of 10 items has `n = 10`; a list of 1,000 items has `n = 1000`.

It does not simply mean actual seconds. It describes how the number of operations grows as the input grows.

9. Big O notation

Big O describes an asymptotic upper bound on an algorithm's growth rate; it is commonly used to report worst-case complexity.

Complexity	Name / idea	Example
`O(1)`	Constant work	Access `numbers[0]`
`O(log n)`	Logarithmic: input shrinks by a factor per step	Binary search on sorted data
`O(n)`	Linear: proportional to input size	Visit each list element
`O(n log n)`	Linearithmic	Merge sort; average/typical efficient comparison sorting
`O(n²)`	Quadratic: compare many pairs / nested loops	Two full nested loops
`O(2^n)`	Exponential growth	Some brute-force subset problems
`O(n!)`	Factorial growth	Exploring every permutation

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$$

This is the typical order from more scalable to less scalable asymptotic growth, not a guarantee about exact running time for every small input.

```
# O(1): constant-time indexed access
print(numbers[0])

# O(n): visit every item
for x in numbers:
    print(x)

# O(n^2): each item paired with all items
for i in numbers:
    for j in numbers:
        print(i, j)
```

10. How to calculate time complexity

One loop

```
for i in range(n):
    print(i)
```

The loop runs `n` times → **`O(n)`**.

Two independent loops

```
for i in range(n):
    print(i)
for j in range(n):
    print(j)
```

Total work is `n + n = 2n`. Ignore constant multipliers → **`O(n)`**.

Nested loops

```
for i in range(n):
    for j in range(n):
        print(i, j)
```

The inner loop runs `n` times for each of `n` outer iterations: `n × n = n²` → **`O(n²)`**.

Halving input

```
while n > 1:
    n //= 2
```

The value is repeatedly halved → **`O(log n)`**.

Different loop sizes

```
for i in range(n):
    for j in range(m):
        print(i, j)
```

The outer loop runs `n` times and the inner loop `m` times → **`O(nm)`**, not always `O(n²)`.

11. Best, average and worst cases

Linear search checks elements one by one.

```
def search(numbers, target):
    for i in range(len(numbers)):
        if numbers[i] == target:
            return i
    return -1
```

Case	Example	Time
Best	Target is first element	`O(1)`
Average	Typically checks a portion of the list	`O(n)`
Worst	Target is last or absent	`O(n)`

The same algorithm can perform differently on different inputs. State which case you are analysing.

12. Space complexity

Space complexity measures how memory use grows with input size.

```
def count_digits(n):
    count = 0
    while n > 0:
        count += 1
        n //= 10
    return count
```

Only a fixed number of extra variables are used → auxiliary space **`O(1)`**.

```
def copy_list(numbers):
    result = []
    for num in numbers:
        result.append(num)
    return result
```

The new list grows with input size → auxiliary space **`O(n)`**.

- **Auxiliary space:** extra memory used by the algorithm, excluding the input itself.
- **Total space:** input storage plus auxiliary memory, depending on the convention used.

13. Big O, Omega and Theta

Notation	Meaning
`O(f(n))`	Asymptotic upper bound
`Ω(f(n))`	Asymptotic lower bound
`θ(f(n))`	Tight asymptotic bound

Example: a simple loop that always visits every list element performs linear work in both its best and worst cases, so it is `θ(n)`. "Big O means worst case" is a common shortcut, but precisely, Big O is an upper-bound notation and can describe other kinds of bounds too.

14. Recursion and the call stack

Recursion occurs when a function calls itself to solve a smaller version of the same problem.

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)

# factorial(4) = 4 × 3 × 2 × 1 = 24
```

- **Base case:** stops recursive calls.
- **Recursive case:** calls the function on a smaller problem.
- Without a suitable base case, calls may continue until Python raises a recursion-depth error.

Factorial takes `O(n)` time and `O(n)` recursive call-stack space. Recursion is useful in trees, divide and conquer, and backtracking.

PART 3

Core data structures

15. Array and Python list

An array is an indexed collection, typically using contiguous storage in a fixed-size array model. A Python list is a dynamic array of object references.

Python list operation	Typical complexity
Access by index	`O(1)`
Search for value	`O(n)`
Append at end	`O(1)` amortized
Insert at beginning	`O(n)`
Delete from beginning	`O(n)`
Sort using built-in sort	`O(n log n)` worst case

Use a list when ordered data and indexed access are useful.

16. Linked list

A linked list stores data in nodes. Each node contains data and a reference (link) to another node.

```
head → [data | next] → [data | next] → [data | None]
```

- **Singly linked:** each node points to the next.
- **Doubly linked:** each node points to both previous and next.
- **Circular linked:** the last node links back to the first or another node, depending on the design.

Operation (singly linked list)	Complexity
Access position `i`	`O(n)`
Search by value	`O(n)`
Insert at head	`O(1)`
Delete head	`O(1)`
Insert after a known node	`O(1)`

A linked list can be useful when node-based insertion/deletion matters. Unlike an array, it does not offer constant-time indexed access.

17. Stack (LIFO)

A stack follows **LIFO: Last In, First Out**. Imagine a stack of plates: the last plate placed on top is the first one removed.

```
TOP → [20]  ← pop removes this first
              [10]
              [ 5]

push = add • pop = remove top • peek/top = inspect top
```

```
stack = []
stack.append(10)  # push
stack.append(20)
print(stack[-1])  # peek: 20
print(stack.pop())  # pop: 20
```

Push and pop at the end of a Python list take `O(1)` amortized time. Uses: undo operations, expression evaluation, balanced brackets and depth-first search.

18. Queue (FIFO)

A queue follows **FIFO: First In, First Out**. The first element inserted is the first removed.

```
dequeue ← FRONT [10] [20] [30] REAR ← enqueue
          First in is removed first.
```

Operations: **enqueue** adds at the rear; **dequeue** removes from the front; **front** inspects the first element.

```
from collections import deque
queue = deque()
queue.append(10)
queue.append(20)
print(queue.popleft())  # 10
```

`collections.deque` is appropriate for efficient queues. Appending and removing from either end take `O(1)`. Uses: scheduling, buffering and breadth-first search.

19. Hash table, set and dictionary

A hash table uses a **hash function** to help map keys to storage locations. Python dictionaries and sets are hash-table-based structures.

- **Hash function:** maps a key to a hash value.
- **Collision:** different keys can map to the same location.
- **Collision handling:** allows colliding keys to be stored and retrieved correctly.
- **Load factor:** measure related to how full a hash table is.

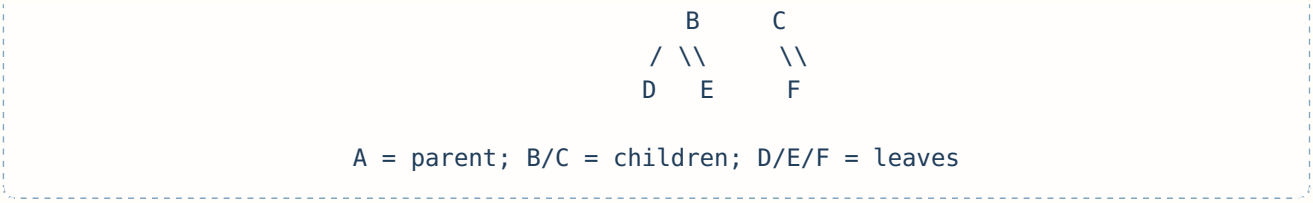
Dictionary and set lookup/membership is usually `O(1)` on average, but can be `O(n)` in the worst case.

Common uses: frequency counting, duplicate detection, Two Sum, fast key-based lookup.

20. Trees

A tree is a hierarchy of nodes connected by edges.

```
Root
  A
/   \\
```



- **Root:** top node.
- **Parent / child:** connected nodes above / below one another.
- **Leaf:** node with no children.
- **Depth:** number of edges from root to a node.
- **Height:** number of edges on the longest downward path from that node to a leaf under the common edge-based convention.
- **Subtree:** a node and its descendants.

Types: binary tree (at most two children); binary search tree / BST (ordered keys); balanced tree (controlled height); heap (priority relation between parent and child); trie (organizes sequences, often strings, by prefixes).

Binary search tree (BST)

In a standard BST with distinct keys, left-subtree values are smaller and right-subtree values are larger than the node. Searching is `O(log n)` in a balanced BST but may degrade to `O(n)` in a highly unbalanced BST.

21. Heap and priority queue

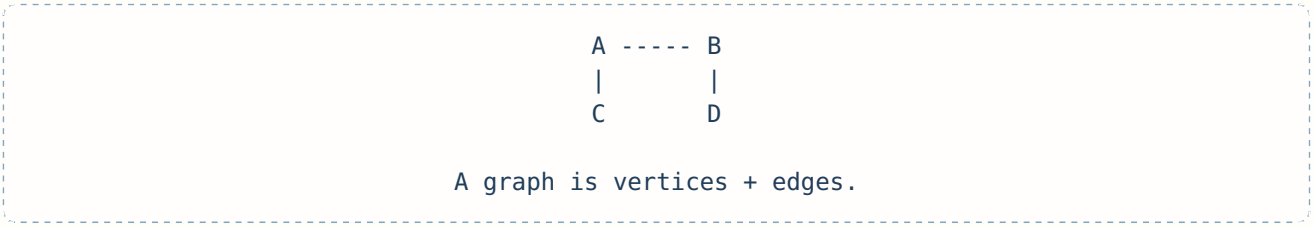
A heap is a tree-based structure that efficiently identifies a highest-priority element. A **min-heap** has the smallest element at the root; a **max-heap** has the largest element there.

Binary heap operation	Typical time
Peek at min/max	`O(1)`
Insert	`O(log n)`
Remove root	`O(log n)`
Build from `n` elements	`O(n)`

Python's `heapq` provides a min-heap. Uses: scheduling, top-`k` problems and Dijkstra's shortest-path algorithm.

22. Graph

A graph consists of **vertices (nodes)** and **edges (connections)**.



- **Directed:** edges have direction.
- **Undirected:** edges have no direction.
- **Weighted:** edges have associated weights.
- **Unweighted:** edges are treated as having equal cost.
- **Cyclic:** contains a cycle.

- **Acyclic:** contains no cycle.

Representing a graph

Adjacency list: store each vertex's neighbors.

```
graph = {
    "A": ["B", "C"],
    "B": ["A", "D"],
    "C": ["A"],
    "D": ["B"]
}
```

Adjacency matrix: a 2D matrix indicates whether an edge exists or stores its weight. An adjacency list is often more memory-efficient for sparse graphs. An adjacency matrix permits constant-time edge-existence checks.

PART 4

Essential algorithms

23. Searching

Searching means finding a target value in a collection.

Linear search

- Checks values one by one.
- Works on sorted and unsorted data.
- Worst-case time: `O(n)`.
- Auxiliary space: `O(1)` for an iterative version.

Binary search

- Repeatedly halves the search range.
- Requires sorted data, or another suitable monotonic search condition.
- Time: `O(log n)`.
- Auxiliary space: `O(1)` for the iterative version.

```
def binary_search(numbers, target):
    left = 0
    right = len(numbers) - 1

    while left <= right:
        mid = (left + right) // 2
        if numbers[mid] == target:
            return mid
        elif numbers[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1
```

24. Sorting algorithms

Sorting arranges data, usually in ascending or descending order.

Algorithm	Best	Average	Worst
Bubble sort*	`O(n)`	`O(n²)`	`O(n²)`
Selection sort	`O(n²)`	`O(n²)`	`O(n²)`
Insertion sort	`O(n)`	`O(n²)`	`O(n²)`
Merge sort	`O(n log n)`	`O(n log n)`	`O(n log n)`
Quicksort	`O(n log n)`	`O(n log n)` expected	`O(n²)`
Heap sort	`O(n log n)`	`O(n log n)`	`O(n log n)`

*Bubble sort is best-case `O(n)` only with an early-exit check; without that optimisation the best case remains `O(n²)`.

- **Bubble sort:** repeatedly swaps adjacent out-of-order elements.
- **Selection sort:** selects the minimum remaining element.
- **Insertion sort:** inserts each element into the sorted portion.
- **Merge sort:** divides, sorts halves and merges them.
- **Quicksort:** partitions values around a pivot.
- **Heap sort:** repeatedly extracts an extreme element using a heap.

Python's `sorted()` and `.sort()` use **Timsort**, a stable sorting algorithm with `O(n log n)` worst-case time.

25. Two pointers

The two-pointer technique uses two indices to solve a problem efficiently. The example below expects a sorted list and checks whether a pair sums to a target.

```
def two_sum_sorted(numbers, target):
    left = 0
    right = len(numbers) - 1

    while left < right:
        total = numbers[left] + numbers[right]
        if total == target:
            return True
        elif total < target:
            left += 1
        else:
            right -= 1
    return False
```

Time: `O(n)`. Auxiliary space: `O(1)`. If the sum is too small, moving left right increases the sum; if too large, moving right left decreases it. Common uses: pair sums, reversing arrays, palindrome checks and removing duplicates from sorted arrays.

26. Sliding window

Useful for contiguous subarrays or substrings. Instead of recalculating every window from scratch, update the current window as it moves.

```
def max_window_sum(numbers, k):
    if k <= 0 or k > len(numbers):
        return None

    current = sum(numbers[:k])
    best = current
    for i in range(k, len(numbers)):
        current += numbers[i] - numbers[i - k]
        best = max(best, current)
    return best
```

Finds the largest sum of a contiguous window of length `k`. Time: `O(n)`; auxiliary space: `O(1)`. Re-summing every window from scratch would take `O(nk)`.

27. Prefix sums

A prefix sum stores cumulative totals so range-sum queries can be answered quickly.

For `[2, 4, 6, 8]`, the prefix sums are `[2, 6, 12, 20]`.

$$sum(l, r) = P[r] - P[l - 1] \text{ (when } l > 0)$$

If `l = 0`, the answer is simply `P[r]`. Building the prefix array takes `O(n)` time and `O(n)` extra space; each range query then takes `O(1)`.

28. Recursion and divide and conquer

Divide and conquer solves a problem by dividing it into smaller subproblems, solving those subproblems and combining their results. Examples: merge sort and many quicksort implementations.

Recursion is one way to implement divide and conquer, but the concepts are not identical: recursion describes a function calling itself; divide and conquer describes a problem-solving strategy.

29. Backtracking

Backtracking builds a possible solution step by step and abandons a path when it cannot lead to a valid solution.

Examples: generating permutations, Sudoku, exploring maze paths, and finding combinations that meet constraints. Backtracking can have exponential time complexity because it may explore many combinations. Pruning invalid paths early can reduce work.

30. Greedy algorithms

A greedy algorithm chooses what seems to be the best immediate option at each step.

Examples: activity selection, fractional knapsack, some minimum-spanning-tree algorithms, and Dijkstra's algorithm for graphs with nonnegative edge weights.

> A greedy choice is not always globally optimal. First establish that the problem has the required properties before choosing a greedy approach.

31. Dynamic programming (DP)

Dynamic programming solves problems by reusing solutions to overlapping subproblems.

- **Memoization:** top-down recursion with caching.
- **Tabulation:** bottom-up calculation using a table.

Fibonacci recurrence: `F(n) = F(n-1) + F(n-2)`. Naive recursive Fibonacci takes exponential time. Bottom-up DP computes each value once: `O(n)` time and `O(n)` space, or `O(1)` extra space if only the last two values are retained.

Common DP problems: knapsack, longest common subsequence, coin change and many optimization problems.

32. Graph traversal: BFS and DFS

- **BFS (breadth-first search):** explores level by level, usually with a queue.
- **DFS (depth-first search):** explores one path deeply before backtracking, using recursion or a stack.

With an adjacency-list representation, both typically take `O(V + E)` time. BFS finds shortest paths by number of edges in an unweighted graph. DFS is useful for connectivity, cycle detection and exploring components.

33. Shortest paths and minimum spanning trees

Algorithm	Main purpose / condition
Dijkstra	Shortest paths from a source when edge weights are nonnegative.
Bellman-Ford	Handles negative edge weights and can detect reachable negative-weight cycles.
Floyd-Warshall	Shortest paths between all pairs of vertices.
Kruskal	Builds a minimum spanning tree by considering edges in increasing weight order.
Prim	Grows a minimum spanning tree from a starting vertex.

A shortest-path tree and a minimum spanning tree solve different problems. An MST minimizes the total weight of its selected edges; it does not necessarily give the shortest route between every pair of vertices.

PART 5

Problem patterns for DSA exams

34. Number-based problems

Problem	Main concept / logic
Even or odd	Modulus `%`; even if remainder on division by 2 is 0.
Reverse a number	Extract with `% 10`, remove with `// 10`, append digit to reversed value.
Count digits	Repeatedly divide by 10; handle zero as a special case if using a loop.
Palindrome number	Reverse and compare with original.
Armstrong number	Extract digits and sum each digit raised to the number of digits.
Prime number	Test possible divisors up to `√n`, with small cases handled separately.
Factors	Check divisibility; for positive integer `n`, `i` is a factor when `n % i == 0`.
GCD	Euclidean algorithm repeatedly replaces pair `(a, b)` with `(b, a % b)`.
LCM	For nonzero integers, `LCM(a,b) = abs(a*b) // GCD(a,b)`.
Factorial	Loop or recursion; product from 1 to `n`; `0! = 1`.
Fibonacci	Iteration, recursion or dynamic programming.

> **Why test divisors only up to √n?** If a number has a factor larger than its square root, it must have a corresponding factor smaller than the square root. So checking divisors through `√n` is sufficient for primality.

35. Array problems

Practise these patterns and learn more than one approach where possible:

- Find minimum and maximum.
- Find the second-largest distinct element.
- Reverse an array.
- Remove duplicates.
- Count frequencies using a dictionary.
- Find a missing number.
- Find duplicate elements.
- Merge two sorted arrays.
- Rotate an array.
- Find a pair with a target sum.

Different solutions may have different time and space costs. For example, a set can simplify duplicate detection but requires extra memory.

36. String problems

Practise reversing a string, checking a palindrome, counting vowels and consonants, counting character frequencies, checking anagrams, finding the first non-repeating character, removing duplicate characters and finding the longest substring satisfying a condition.

Useful techniques: dictionaries, sets, sorting, two pointers and sliding windows.

37. Bit manipulation

Bit manipulation operates on the binary representation of integers.

Operator	Meaning
`&`	Bitwise AND
`|`	Bitwise OR
`^`	Bitwise XOR
`~`	Bitwise NOT
`<<`	Left shift
`>>`	Right shift

For nonnegative integers, `n & 1` checks odd/even: result 1 means odd, 0 means even. XOR is helpful in selected problems involving pairs and missing values.

PART 6

Interview and correctness concepts

38. Abstract Data Types (ADTs)

An ADT describes **what operations** a data structure supports without specifying exactly how it is implemented.

Example: the stack ADT supports push, pop and peek. It can be implemented using an array/list or a linked list. This separates behaviour from implementation.

39. Stable and in-place sorting

- **Stable sort:** preserves the relative order of equal-key elements.
- **In-place algorithm:** uses little extra memory beyond the input, according to the algorithm's convention.

Python's built-in sort is stable. This helps when sorting records by multiple fields in stages. Stability and in-place behaviour are separate properties: an algorithm can have one without the other.

40. Amortized analysis

Amortized analysis considers the cost per operation over a sequence of operations, even if a particular operation can be expensive.

Appending to a Python list is `O(1)` amortized. Sometimes its underlying storage must grow and elements must be copied; that occasional cost is spread across many append operations.

Amortized complexity is not the same as average-case complexity over randomly selected inputs.

41. Invariants and correctness

An **invariant** is a property that remains true throughout an algorithm. To reason about a loop:

1. **Initialization:** the invariant holds before the loop begins.
2. **Maintenance:** each iteration preserves it.
3. **Termination:** when the loop ends, the invariant helps establish correctness.

For binary search, a key invariant is: if the target exists, it remains inside the current search range. This kind of reasoning helps find bugs and explain solutions in interviews.

42. Constraints and edge cases

Before coding, ask:

- Can the input be empty?
- Can numbers be negative?
- Can values repeat?
- Is the list already sorted?
- Can the input contain zero?
- What should happen when no solution exists?
- How large can `n` be?

Constraints help choose an algorithm. An `O(n²)` solution might be fine for a few hundred elements but unsuitable for a million elements.

FINAL REVISION

DSA checklist & study order

Python foundations

- [ ] Variables, data types and operators
- [ ] Conditions and loops
- [ ] Functions, parameters, arguments and return
- [ ] Lists, strings, tuples and sets
- [ ] Dictionaries, keys, values and `.items()`
- [ ] Mutable versus immutable objects

Complexity

- [ ] Big O, Omega and Theta
- [ ] Best, average and worst cases
- [ ] Time complexity of loops
- [ ] Auxiliary space and total space
- [ ] Recursion and call-stack space
- [ ] Amortized analysis

Data structures

- [ ] Arrays and linked lists
- [ ] Stacks and queues
- [ ] Hash tables and dictionaries
- [ ] Trees, BSTs and heaps
- [ ] Graphs and graph representations

Algorithms

- [ ] Linear and binary search
- [ ] Sorting algorithms
- [ ] Two pointers and sliding window
- [ ] Prefix sums
- [ ] Recursion and divide and conquer
- [ ] Backtracking, greedy and dynamic programming
- [ ] BFS, DFS and shortest paths

Practice problems

- ☐ Even/odd, voting eligibility and factors
- ☐ Reverse, digit count, palindrome and Armstrong
- ☐ Prime, GCD, LCM and factorial
- ☐ Second largest and frequency counting
- ☐ Array and string problems
- ☐ Basic bit manipulation

Recommended study order

1. Python fundamentals → **2.** Time and space complexity → **3.** Arrays, strings and dictionaries → **4.** Searching and sorting → **5.** Recursion → **6.** Common problem-solving patterns → **7.** Trees and graphs → **8.** Greedy and dynamic programming.

For each problem, be ready to explain what the algorithm does, why it works, its time complexity, its extra-space complexity and the edge cases it handles.

Understand the logic. Trace it. Analyse it. Practise it.